

Estrutura de Dados & Algoritmos

Desenvolvendo Código Limpo e Eficiente em TypeScript

Pressione barra de espaço para começar →



Agenda da Aula de Hoje

O que vamos aprender no encontro de hoje:

</> 1. Conceitos Fundamentais

Tipagem estática, imutabilidade e escopo de variáveis no TypeScript.

🔗 2. Fluxo de Execução

Visualização em diagrama do ciclo de vida de uma requisição.

🔄 3. Refatoração & Clean Code

Comparação prática: Códigos ruins vs. Boas práticas.

📁 4. Desafio Prático

Exercício hands-on para fixação do conhecimento.



Conceito: Tipagem e Interfaces

Interfaces definem o contrato para objetos em nossa aplicação.

-  Segurança em tempo de compilação: Evita erros comuns de *undefined* ou *null*.
-  Documentação viva: O próprio editor fornece autocomplete e auto-documentação.
-  Reutilização: Podem ser estendidas ou compostas para criar estruturas complexas.

Dica do Professor: Sempre prefira declarar interfaces para contratos públicos da sua API.



Exemplo Prático: Animação de Código Passo a Passo

Use os botões de navegação para avançar no código linha por linha:

```
1  // 1. Definição da Interface do Usuário
2  interface User {
3      id: number;
4      name: string;
5      role: 'admin' | 'student';
6  }
7
8  // 2. Função de validação
9  function isAdmin(user: User): boolean {
10     return user.role === 'admin';
11 }
12
13 // 3. Execução e teste
14 const currentUser: User = {
15     id: 101,
16     name: "Ana Silva",
17     role: "student"
18 };
19
```



Comparando Códigos (Refatoração)

Veja a diferença entre um código confuso e uma versão limpa:



Mau Exemplo (Antigo)

```
1  function process(u: any) {  
2    if (u.a == 1) {  
3      if (u.status == 'A') {  
4        return true;  
5      } else {  
6        return false;  
7      }  
8    }  
9    return false;  
10 }  
11
```



Bom Exemplo (Refatorado)

```
1  interface Account {  
2    isAdmin: boolean;  
3    isActive: boolean;  
4  }  
5  
6  function isValidAccount(account: Account): boolean {  
7    return account.isAdmin && account.isActive;  
8  }  
9
```



Arquitetura do Sistema (Diagrama)

Podemos desenhar fluxos diretamente usando Markdown com Mermaid:





Desafio Prático para os Alunos

```
1  interface Student {
2      name: string;
3      grade: number;
4  }
5
6  // TODO: Implemente a função abaixo
7  function getApprovedStudents(students: Student[]): string[] {
8      // Sua solução aqui...
9  }
10
```

? Quiz de Fixação



Recursos Adicionais e Links

Para aprofundar seus estudos após a aula:

Obrigado pela atenção! 🎉

Coluna da Esquerda

Conteúdo ou explicação do lado esquerdo.

- Suporte completo a Markdown
- Estilização rápida via Tailwind CSS
- Animação simples de itens

Coluna da Direita

Conteúdo complementar ou imagem do lado direito.

```
1 // Exemplo rápido no lado direito
2 const status = "Pronto";
3 console.log(`Estado: ${status}`);
4
```

Destaque Dinâmico de Código

Navegue linha por linha utilizando as etapas de visualização:

```
1  // Etapa 1: Definição de interface
2  interface Projeto {
3      nome: string;
4      versao: number;
5  }
6
7  // Etapa 2: Instanciação
8  const meuProjeto: Projeto = {
9      nome: "Slidev Docs",
10     versao: 1.0
11 };
12
13 // Etapa 3: Execução
14 console.log(meuProjeto.nome);
15
```

Animações de Elementos (`v-click`)

Exiba tópicos um a um conforme você clica nos slides:

Fórmulas Matemáticas (KaTeX)

Você pode renderizar equações complexas nativamente:

$$\mathcal{L}\{f(t)\} = F(s) = \int_0^{\infty} f(t)e^{-st} dt$$

E alinhar equações inline como $E = mc^2$.

$$\mathcal{L}\{f(t)\} = F(s) = \int_0^{\infty} f(t)e^{-st} dt$$

Perguntas & Respostas

Obrigado pela atenção!

Documentação do Slidev · GitHub